

RideSmart

Henrique Eduardo Costa Da Silva

*Departamento de Engenharia de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil*

João Victor Moura Lucas da Silva

*Departamento de Engenharia de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil*

Murilo de Lima Barros

*Departamento de Engenharia de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil*

Ramon Vinícius Ferreira de Souza

*Departamento de Engenharia de Computação e Automação
Universidade Federal do Rio Grande do Norte
Natal, Brasil*

I. INTRODUÇÃO

O avanço das plataformas de mobilidade urbana sob demanda trouxe à tona a necessidade de otimização contínua de rotas e tempos de resposta. Modelos tradicionais de roteamento consideram o embarque estritamente no ponto inicial do usuário. No entanto, o projeto RideSmart aborda um cenário mais flexível e realista: o usuário aceita caminhar uma distância máxima X a partir de uma origem A até um ponto de embarque otimizado P , visando mitigar congestionamentos, contornar barreiras geométricas ou acessar vias de tráfego rápido, reduzindo o tempo ou a distância total de deslocamento até o destino final B .

II. MODELAGEM DO PROBLEMA

A. Representação Teórica e Estrutura de Dados

O problema de mobilidade urbana é modelado através de um grafo direcionado e ponderado $G = (V, E)$, extraído através da biblioteca OSMnx. Ele é composto pelas seguintes estruturas:

- Nós (V): Representam as interseções viárias (cruzamentos) e pontos finais de vias. Cada nó possui coordenadas geográficas associadas, que são: latitude e longitude.
- Arestas (E): Representam os segmentos de vias urbanas transitáveis, como ruas e avenidas. As arestas são direcionadas, respeitando a mão de direção das ruas.

B. Definição de Pesos (W)

Dependendo do modo de operação definido pelos argumentos do programa (–traffic), o peso associado a cada aresta assume funções de custo distintas:

- length: O peso representa a distância métrica real em metros (m) da rua.
- travel_time: Calculado pelo OSMnx a partir da velocidade máxima da via e de sua extensão, sem interferência de tráfego.
- travel_time_with_traffic: Incorpora um modelo estocástico de atraso. O tempo base é acrescido de uma variável aleatória baseada na distribuição exponencial (random.expovariate), cuja intensidade é escalada caso a via seja uma rodovia/via expressa (motorway, trunk,

primary) ou o horário seja de pico (peak), penalizando vias principais.

C. Heurística de Pontos de Embarque

Para evitar o aglomeramento de pontos candidatos a embarque em lugares muito próximos e na mesma direção, foi implementado uma espécie de filtro angular.

Dado o raio de caminhada X , pontos aleatórios são mapeados para os nós mais próximos da malha viária (ox.distance.nearest_nodes). O algoritmo calcula o ângulo θ formado pelo vetor $\overrightarrow{AP}$. Um novo ponto só é aceito se a distância angular para todos os pontos já aceitos for maior ou igual ao limite –min-angle, gerando uma dispersão radial uniforme ao redor da origem.

III. ALGORITMOS UTILIZADOS

Foram implementados os seguintes algoritmos de caminhos mínimos: Dijkstra Simples, Dijkstra com Heap, A* (A-Star) e SPFA (Shortest Path Faster Algorithm).

A. Dijkstra Simples

O Dijkstra Simples atua como a abordagem de referência base para a resolução do problema de caminhos mínimos, operando através de uma estratégia de exploração exaustiva e não-informada do espaço de estados. A sua lógica consiste em determinar a rota ideal a partir de sucessivas iterações onde, a cada passo, o algoritmo realiza uma busca linear varrendo sistematicamente todos os vértices do grafo para identificar aquele que possui a menor distância provisória acumulada. Por não utilizar estruturas de dados avançadas para gerir a prioridade dos nós, essa varredura direta resulta numa complexidade computacional de pior caso de $O(V^2)$, onde V representa o número total de vértices.

B. Dijkstra com Heap

O Dijkstra com Heap é uma evolução do Dijkstra Simples, substituindo a busca linear por uma estrutura de dados baseada em uma fila de prioridades, implementada por uma Min-Heap. Essa modificação otimiza a operação mais frequente do algoritmo: a extração sistemática do nó que apresenta

o menor custo provisório acumulado até o momento. Ao estruturar os vértices candidatos nessa árvore balanceada, o custo computacional para selecionar o próximo cruzamento a ser expandido cai para um tempo logarítmico, o que reduz a complexidade total de tempo para $O((V + E) \log V)$, onde V é o número de vértices e E é o número de arestas.

C. A* (A-Star)

Diferente do algoritmo Dijkstra (que expande a busca de forma radial e uniforme em todas as direções), o A* introduz um conceito de busca guiada. Ele utiliza o conhecimento da posição geográfica do destino B para priorizar os nós que parecem estar logicamente mais próximos do objetivo.

Para decidir qual nó expandir a seguir, o A* calcula uma função de avaliação $f(n)$ para cada nó candidato n :

$$f(n) = g(n) + h(n) \quad (1)$$

onde:

- $g(n)$ (Custo Acumulado): É o custo real exato do caminho percorrido desde o ponto de embarque P até o nó atual n .
- $h(n)$ (Heurística): É uma estimativa do custo restante para ir do nó atual n até o destino final B . Como o algoritmo não conhece o caminho real antes de explorá-lo, ele usa a geometria para estimar essa distância em linha reta.

As seguintes métricas geométricas foram implementadas:

- Círculo Máximo (Haversine): Baseia-se na geometria esférica para calcular a menor distância entre dois pontos à superfície da Terra, utilizando o raio médio do planeta. Por considerar explicitamente a curvatura terrestre, esta função é boa para coordenadas geográficas reais, uma vez que a linha reta sobre a esfera representa o menor trajeto físico possível no espaço tridimensional.
- Euclidiana Planar: Projeta as coordenadas esféricas do mapa num plano cartesiano bidimensional local, utilizando o Teorema de Pitágoras ($\sqrt{\Delta x^2 + \Delta y^2}$). É uma métrica mais eficiente computacionalmente, pois não utiliza funções trigonométricas. Tem um erro pequeno em raios urbanos curtos.
- Manhattan: Calcula o custo remanescente através da soma das diferenças absolutas dos eixos horizontal e vertical ($|\Delta x| + |\Delta y|$), simulando um deslocamento em ângulos retos de 90 graus. Essa métrica funciona bem em regiões que a malha viária segue um padrão quadriculado.

D. SPFA (Shortest Path Fast Algorithm)

O Shortest Path Faster Algorithm (SPFA) funciona como uma estratégia dinâmica de propagação de melhorias para encontrar caminhos mínimos num grafo, estruturada conceitualmente como uma evolução direta do algoritmo clássico de Bellman-Ford. Enquanto o método de Bellman-Ford opera através de uma força-bruta rígida que reavalua cegamente todas as arestas do mapa por $V - 1$ vezes, a lógica do SPFA baseia-se no princípio da seletividade. O algoritmo assume que uma determinada rua ou conexão só pode oferecer um caminho mais rápido se o nó que lhe dá origem tiver sofrido uma

redução de custo recentemente. Assim, em vez de processar a rede inteira de forma repetitiva, o SPFA direciona o foco computacional apenas para os pontos onde uma otimização acabou de acontecer, fazendo com que a descoberta de uma rota eficiente se propague pelo grafo.

A estrutura lógica do algoritmo organiza os nós que sofreram atualizações em uma fila de processamento. Inicialmente, define-se a distância para a origem como zero e para todos os outros destinos como infinito, inserindo o ponto inicial na fila. A cada iteração, o algoritmo retira o nó que está na frente da fila e analisa os seus vizinhos diretos; se a passagem por este nó reduzir o custo para alcançar um vizinho, o caminho desse vizinho é relaxado e ele é colocado na fila para partilhar a nova vantagem com a sua própria vizinhança. Para evitar redundâncias e desperdício de processamento, se um nó atualizado já estiver na fila aguardando a sua vez de expandir, o algoritmo consolida a informação e impede que ele seja inserido em duplicado.

No que diz respeito à complexidade computacional, em redes urbanas e mapas viários reais, que são grafos tipicamente esparsos (onde cada cruzamento possui poucas saídas), o algoritmo converge de forma rápida, operando na prática com uma complexidade média estimada em $O(E)$, onde E é o número de arestas. No entanto, por herdar a estrutura de Bellman-Ford para ser capaz de lidar com pesos negativos e detectar ciclos infinitos, o seu pior cenário teórico permanece em $O(V \cdot E)$, onde V é o número de vértices.

IV. EXPERIMENTOS

Os algoritmos foram testados na malha urbana da cidade de Natal, Rio Grande do Norte. A escolha da cidade justifica-se pelo conhecimento do grupo do traçado viário local, que combina áreas com malhas ortogonais e avenidas de tráfego rápido.

Os experimentos consistiram em simular uma jornada urbana típica, onde o utilizador deseja deslocar-se de uma origem, como a UFRN, até um destino final B , como a Zona Norte da cidade. O sistema elencou 10 pontos de embarque (P) candidatos, localizados dentro de um raio máximo de caminhada de 200 metros a partir da posição inicial do utilizador. Cada algoritmo de menor caminho foi submetido a dois cenários de custo distintos:

- 1) Cenário de Tempo de Viagem Ideal: O custo das arestas (ruas) foi definido em segundos, dividindo o comprimento linear em metros (obtido via OpenStreetMap) pela velocidade máxima permitida da via.
- 2) Cenário de Tempo de Viagem com Tráfego: O custo temporal das arestas foi penalizado dinamicamente através de uma variável aleatória baseada numa distribuição exponencial (simulando os regimes de tráfego normal e peak de Natal).

V. RESULTADOS

Os testes realizados na rota entre a UFRN e a Zona Norte de Natal revelaram que a introdução do trânsito sintético alterou significativamente as rotas sugeridas pelos algoritmos. No

cenário de tempo ideal, as rotas convergiram consistentemente para os eixos viários principais, como a Avenida Salgado Filho e a BR-101. Contudo, com o regime de tráfego estocástico, o custo das vias principais aumentou devido às penalizações exponenciais, forçando os algoritmos a desviarem o trajeto para ruas secundárias e rotas alternativas por bairros adjacentes. Embora, as vezes, a distância física total em metros tenha aumentado, o tempo de viagem global foi consideravelmente menor.

A análise comparativa dos critérios de otimização demonstrou que a rota de menor distância física (comprimento em metros) nem sempre correspondeu à rota mais rápida, especialmente quando o tráfego foi ativado. No cenário de tempo de viagem ideal, o caminho mais curto tendeu a ser o mais rápido devido à ausência de bloqueios.

No que diz respeito à eficiência computacional, o algoritmo A^* expandiu substancialmente menos nós do que o Dijkstra tradicional. Da mesma forma, a implementação do Dijkstra com Heap foi drasticamente mais eficiente do que o Dijkstra simples, reduzindo o tempo de CPU, uma vez que a estrutura de árvore balanceada otimizou a obtenção do nó de menor custo.

Por fim, o algoritmo adaptado da literatura, o SPFA, teve um bom desempenho, encontrando o melhor caminho na casa dos milissegundos. No entanto, o A^* acabou sendo bem mais rápido, sendo em média dezenas de vezes mais rápido que o SPFA, como pode ser visto na tabela 1.

TABLE I
TEMPOS MÉDIOS DE EXECUÇÃO DOS ALGORITMOS

Algoritmo	Tempo de Execução (ms)
A^*	17,29
Dijkstra (Heap)	85,44
SPFA	834,31
Dijkstra	16.390,50

VII. DISCUSSÃO CRÍTICA

A flexibilidade de caminhar até uma determinada distância máxima melhorou a solução global na maioria dos casos testados. Ao permitir que o passageiro se deslocasse a pé para uma rua paralela ou para o sentido oposto de uma avenida congestionada, o algoritmo do veículo pode iniciar a rota já fora de gargalos críticos de trânsito. Essa dinâmica elimina a necessidade de o motorista realizar retornos demorados ou enfrentar os cruzamentos mais saturados, reduzindo o tempo total da viagem.

Por outro lado, os experimentos evidenciaram cenários específicos onde caminhar acabou por atrapalhar ou não trouxe vantagens práticas ao utilizador. Isso ocorre principalmente quando o destino final se encontrava numa direção geometricamente oposta ao ponto de embarque candidato, fazendo com que o ganho de tempo obtido pelo carro não compensasse o tempo gasto pelo passageiro a caminhar os 200 metros. Além disso, onde a malha secundária ao redor do ponto de caminhada também estava com tráfego intenso, a mudança

de embarque resultou em rotas alternativas igualmente lentas, tornando o esforço físico da caminhada inútil.

Do ponto de vista da modelagem, foram identificadas limitações importantes na abordagem proposta. A principal delas reside no fato do trânsito ser gerado de forma sintética por uma distribuição matemática, o que não replica perfeitamente o comportamento histórico e os bloqueios crônicos de pontos específicos da cidade, como a Ponte Newton Navarro, por exemplo. Da mesma forma, o modelo simplificou a rede ao não contabilizar as restrições de sentido de circulação (mãos proibidas), a presença de semáforos nos cruzamentos e a velocidade e disposição real de caminhada do utilizador sob diferentes condições climáticas ou de relevo.

Para aproximar esse modelo de uma aplicação real de mobilidade urbana, seria necessário integrar o sistema a APIs de tráfego em tempo real (como as do Google Maps ou Waze) para alimentar os algoritmos com tempos de viagem dinâmicos. O algoritmo também precisaria ser refinado para incorporar elementos de segurança urbana para evitar sugerir pontos de embarque em áreas de risco em determinados horários, além de considerar restrições de acessibilidade.

VII. CONCLUSÃO

O desenvolvimento e a experimentação do sistema RideSmart na malha viária da cidade de Natal demonstraram que a introdução de flexibilidade no ponto de embarque do passageiro é uma estratégia eficaz para otimizar a mobilidade urbana. Essa abordagem permite viagens em rotas de vias alternativas ou em sentidos de circulação mais fluidos, resultando numa redução significativa do tempo global de viagem combinada (caminhada mais deslocamento automóvel), especialmente sob regimes de tráfego intenso.

Do ponto de vista computacional, tanto o algoritmo A^* , o Dijkstra com Heap e o SPFA foram satisfatórios, tendo tempos de execução rápidos (milissegundos). Apenas o Dijkstra Simples pode ser considerado computacionalmente ineficiente em comparação com os outros, demorando segundos para encontrar o melhor caminho.

Embora a modelagem atual apresente limitações devido ao caráter sintético do trânsito e à simplificação de restrições viárias complexas, o projeto mostra que é possível modificar a abordagem tradicional. Para aproximar o sistema de uma aplicação de produção real, é necessário integrar dados de tráfego em tempo real e dados de segurança, consolidando o conceito de embarque flexível como uma resposta inteligente para os desafios da mobilidade urbana nas grandes cidades.